

Image Segmentation

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 12, 2026

1. Learning Outcomes
2. Introduction
3. Semantic Segmentation
 1. Sliding Window
 2. Convolution
4. Upsampling
5. Transposed Convolution
6. Fully Convolutional Networks (FCNs)
7. Residual Connections and U-Net
8. Instance Segmentation
 1. Mask R-CNN
9. Panoptic Segmentation
10. References

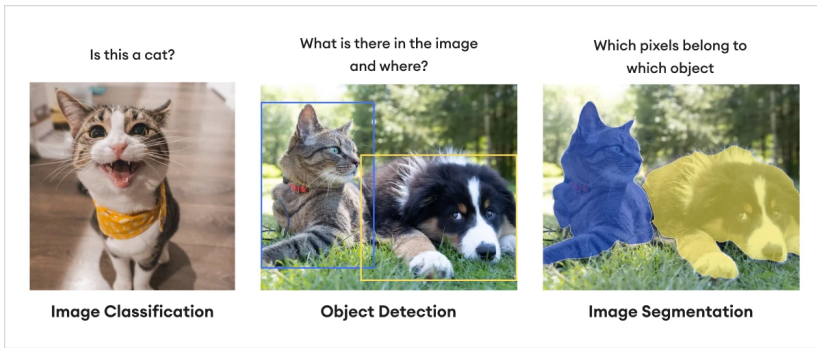
Image Segmentation: **Learning Outcomes**

- ▶ Understand the need and core idea behind image segmentation
- ▶ Explain transposed convolution, Fully Convolutional Networks (FCNs), and U-Net
- ▶ Differentiate between semantic, instance, and panoptic segmentation
- ▶ Identify key challenges and limitations in image segmentation
- ▶ Explore future directions and advanced research topics

Image Segmentation: **Introduction**

What is Image Segmentation?

- **Definition:** Partitioning an image into multiple meaningful segments.
- **Goal:** Assign a label to every pixel such that pixels with the same label share visual characteristics.



Classification



CAT

No spatial extent

Semantic Segmentation



GRASS, CAT,
TREE, SKY

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



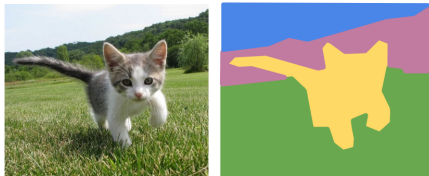
DOG, DOG, CAT

[This image is CC0 public domain](#)

- **Classification:** Identify the main object in an image.
- **Detection:** Locate objects with bounding boxes.
- **Segmentation:** Classify each pixel to delineate object boundaries.

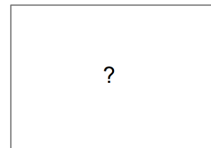
- ▶ **Pixel-level labeling is labor-intensive**
- ▶ Varying object size and shape
- ▶ Occlusion and overlapping objects
- ▶ Class imbalance (background vs. object)

Image Segmentation: **Semantic Segmentation**



GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image,
each pixel is labeled with a semantic category.

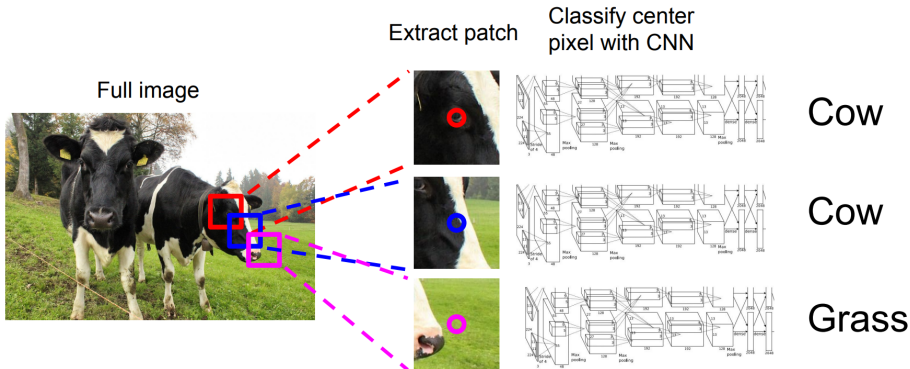


At test time, classify each pixel of a new image.

Full image



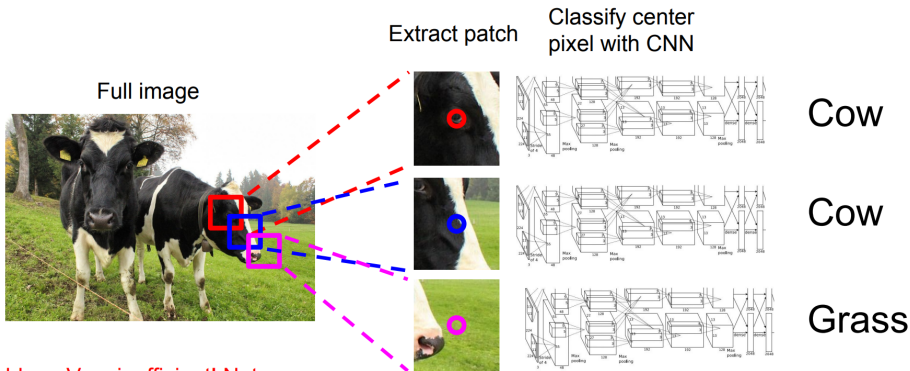
Semantic Segmentation Idea: Sliding Window



Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

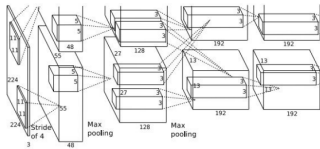
Semantic Segmentation Idea: Sliding Window (cont.)



Problem: Very inefficient! Not reusing shared features between overlapping patches

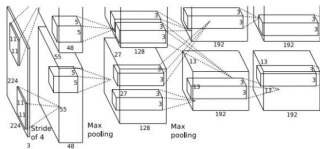
Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

Full image



An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Full image

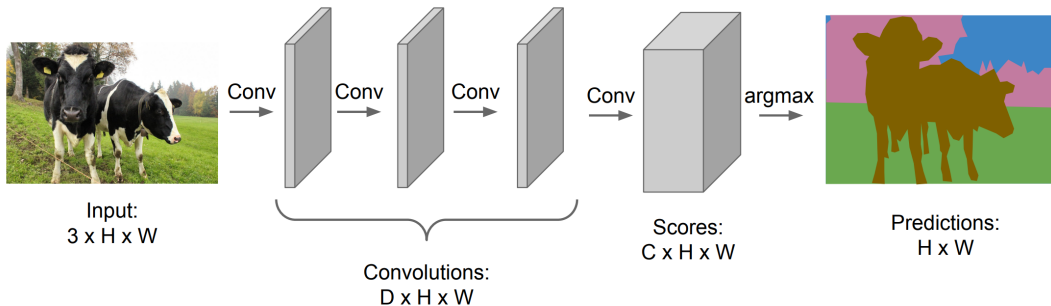


An intuitive idea: encode the entire image with conv net, and do semantic segmentation on top.

Problem: classification architectures often reduce feature spatial sizes to go deeper, but semantic segmentation requires the output size to be the same as input size.

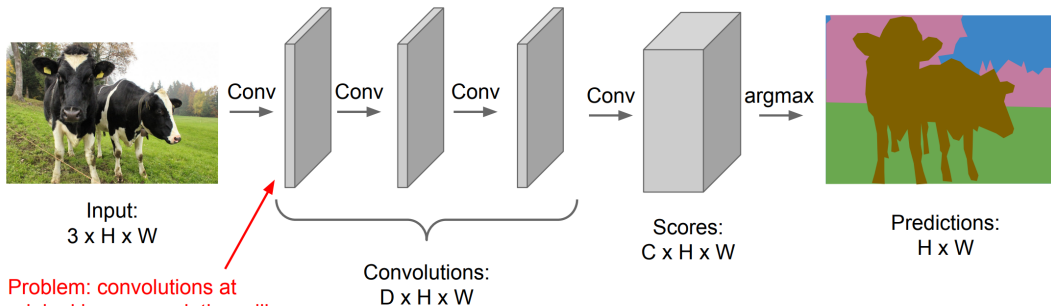
Semantic Segmentation Idea: Fully Convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Semantic Segmentation Idea: Fully Convolutional (cont.)

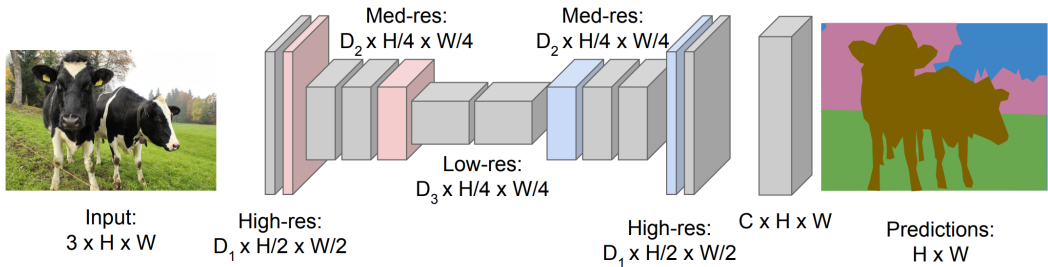
Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional (cont.)

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

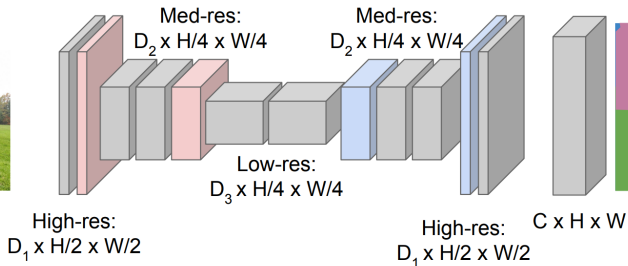
Semantic Segmentation Idea: Fully Convolutional (cont.)

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
???



Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Image Segmentation: **In-Network Upsampling**

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

"Bed of Nails"

1	2
3	4

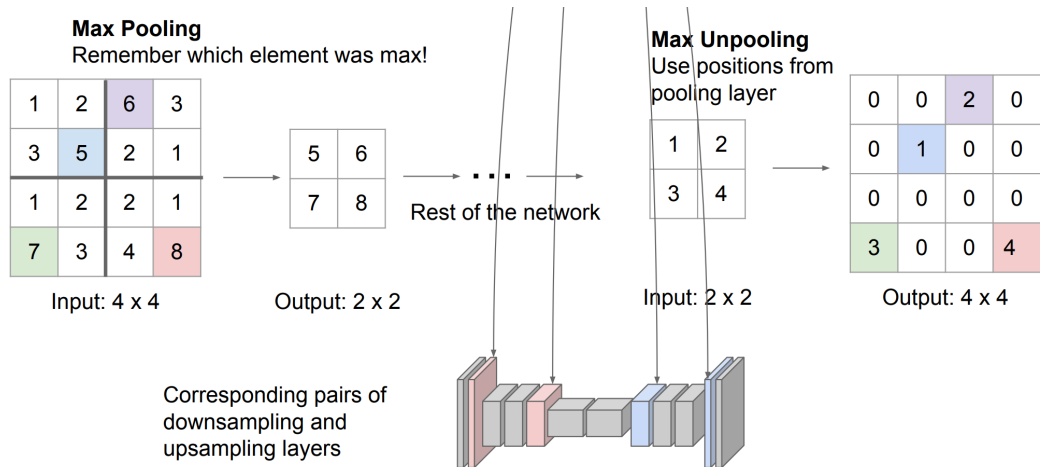


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

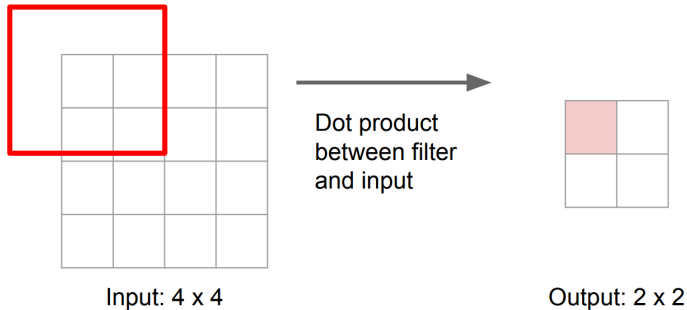
Input: 2 x 2

Output: 4 x 4

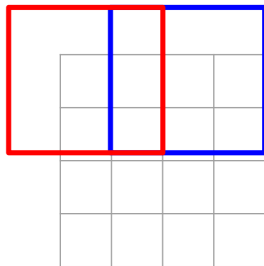
In-Network Upsampling: Max Unpooling



Recall: Normal 3 x 3 convolution, stride 2 pad 1



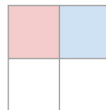
Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4



Dot product
between filter
and input



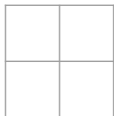
Output: 2 x 2

Filter moves 2 pixels in
the input for every one
pixel in the output

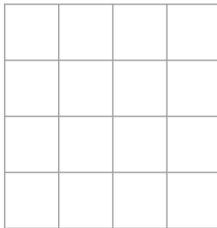
Stride gives ratio between
movement in input and
output

We can interpret strided
convolution as “learnable
downsampling”.

3 x 3 **transposed** convolution, stride 2 pad 1

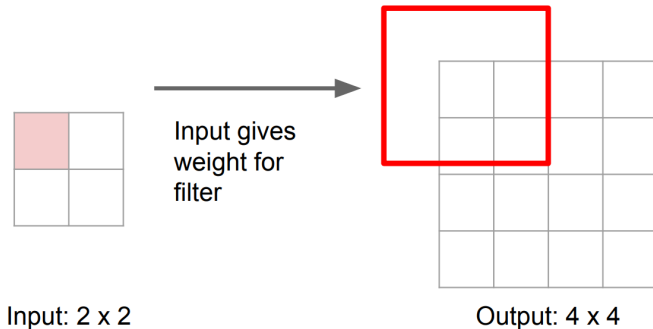


Input: 2 x 2

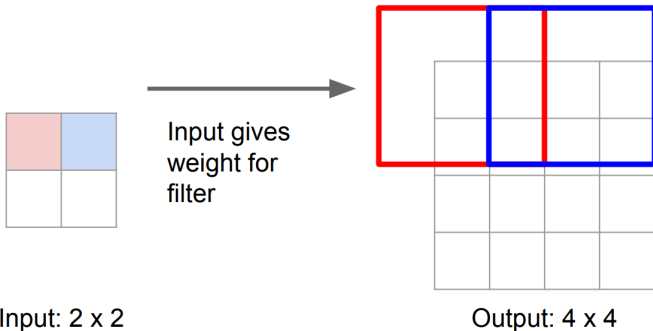


Output: 4 x 4

3 x 3 **transposed** convolution, stride 2 pad 1



3 x 3 **transposed** convolution, stride 2 pad 1

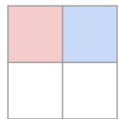


Filter moves 2 pixels in the output for every one pixel in the input

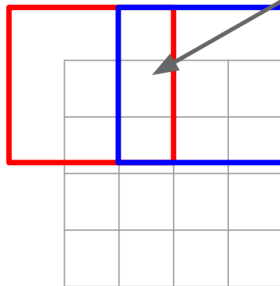
Stride gives ratio between movement in output and input

3 x 3 **transposed** convolution, stride 2 pad 1

Sum where
output overlaps



Input gives
weight for
filter



Filter moves 2 pixels in
the output for every one
pixel in the input

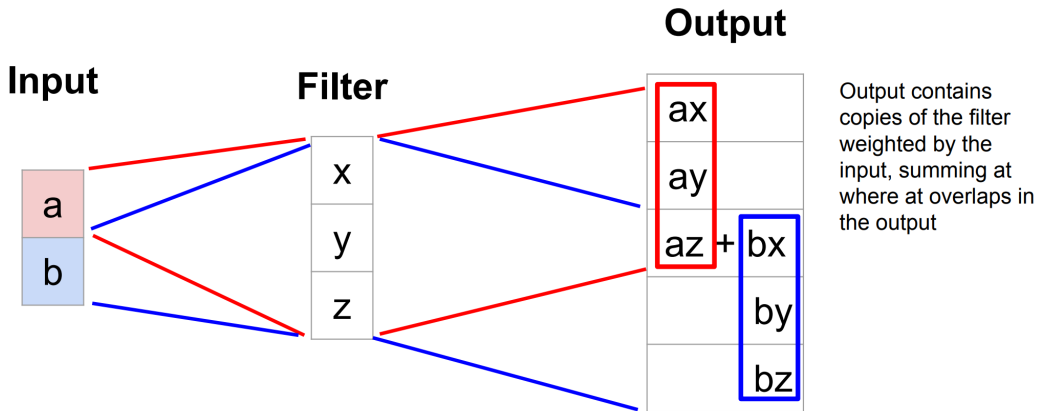
Stride gives ratio between
movement in output and
input

Input: 2 x 2

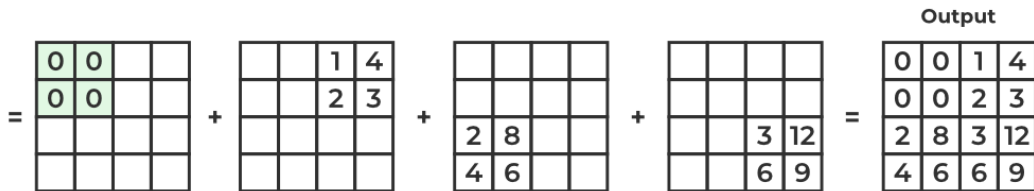
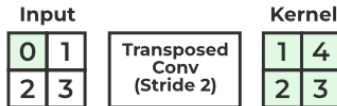
Output: 4 x 4

Image Segmentation: **Transposed Convolution**

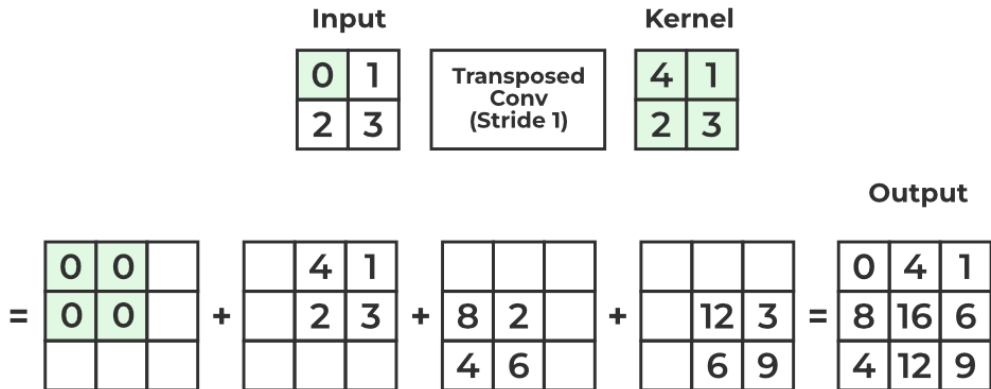
Transposed Convolution: 1D Example



Transposed Convolution: How It Works



Transposed Convolution: How It Works (cont.)



The output $O(x, y)$ of a transposed convolution is computed as:

$$O(x, y) = \sum_{i,j} I(i, j) \cdot K(x - i \cdot s, y - j \cdot s)$$

where:

- ▶ $O(x, y)$ is the output at position (x, y) ,
- ▶ $I(i, j)$ is the input value at (i, j) ,
- ▶ $K(x', y')$ is the kernel value at (x', y') ,
- ▶ s is the stride.

$$\begin{aligned} H_{\text{out}} = & (H_{\text{in}} - 1) \times \text{stride}[0] - 2 \times \text{padding}[0] \\ & + \text{dilation}[0] \times (\text{kernel_size}[0] - 1) \\ & + \text{output_padding}[0] + 1 \end{aligned} \quad (1)$$

$$\begin{aligned} W_{\text{out}} = & (W_{\text{in}} - 1) \times \text{stride}[1] - 2 \times \text{padding}[1] \\ & + \text{dilation}[1] \times (\text{kernel_size}[1] - 1) \\ & + \text{output_padding}[1] + 1 \end{aligned} \quad (2)$$

where:

- ▶ $H_{\text{out}}, W_{\text{out}}$ - Output height and width.
- ▶ $H_{\text{in}}, W_{\text{in}}$ - Input height and width.
- ▶ Stride - Step size of the filter movement.
- ▶ Padding - Number of pixels added around the input.
- ▶ Dilation - Spacing between kernel elements.
- ▶ Kernel size - Size of the convolution filter.

Image Segmentation: **Fully Convolutional Networks (FCNs)**

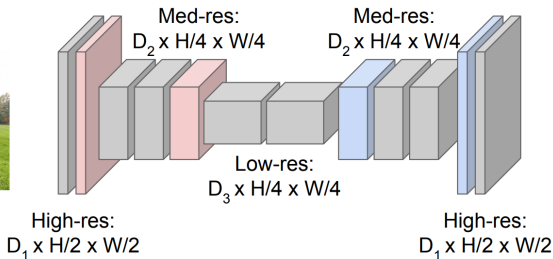
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



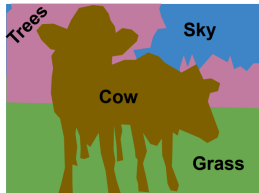
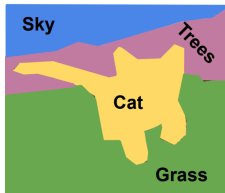
Upsampling:
Unpooling or strided
transposed convolution



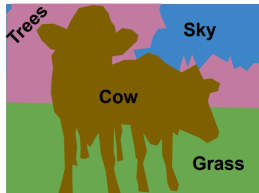
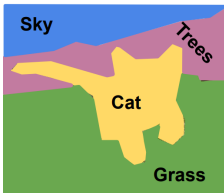
Predictions:
 $H \times W$

Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

- Label each pixel in the image with a category label



- Label each pixel in the image with a category label



- Does not differentiate instances, only care about pixels

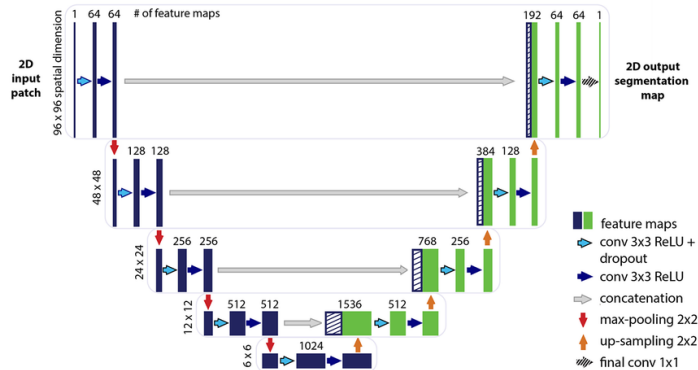
Image Segmentation: **Residual Connections and U-Net**

- ▶ The downsampling-then-upsampling approach works well for semantic segmentation.
- ▶ **But... can we do better?**
- ▶ **Problem:** Important details and spatial information may be lost during downsampling.

- ▶ The downsampling-then-upsampling approach works well for semantic segmentation.
- ▶ **But... can we do better?**
- ▶ **Problem:** Important details and spatial information may be lost during downsampling.
- ▶ **Solution:** Introduce **residual connections** to preserve spatial information.

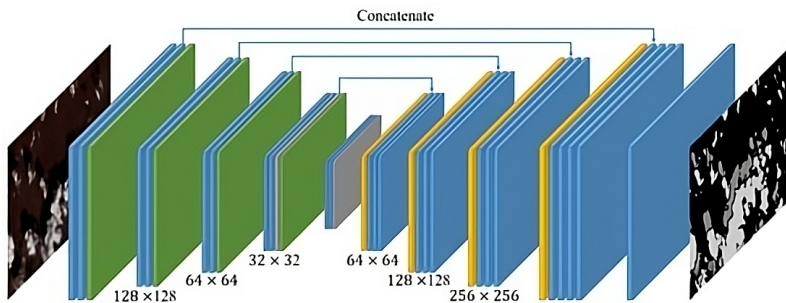
Residual Connections in Segmentation

- ▶ Directly connect features from downsampling layers to upsampling layers.
- ▶ Help recover lost spatial details and improve segmentation accuracy.



- ▶ There are two Types of Residuals:
 - **Addition:** Adds features from the encoder to the decoder **element-wise**.
 - **Concatenation:** Concatenates features from the encoder to the decoder along the **channel dimension**.
- ▶ **Which Is Better?**
 - Concatenation is often better because it retains more feature information from the encoder.
 - **Note:** technically, concatenation might be harder to implement because it requires aligning input and output shapes.

- ▶ This architecture, with residual connections, is called **U-Net**.
- ▶ **Why the name?**
 - The architecture resembles the shape of the letter "U".
 - Features are downsampled in the encoder and upsampled in the decoder, with skip connections in between.
- ▶ U-Net is widely used for segmentation tasks, especially in biomedical imaging.



► Pretrained Encoder:

- The encoder can use a pretrained backbone (e.g., ResNet, EfficientNet).
- This help utilize features learned on large datasets (e.g., ImageNet).
- Only the decoder is trained from scratch for segmentation-specific tasks.

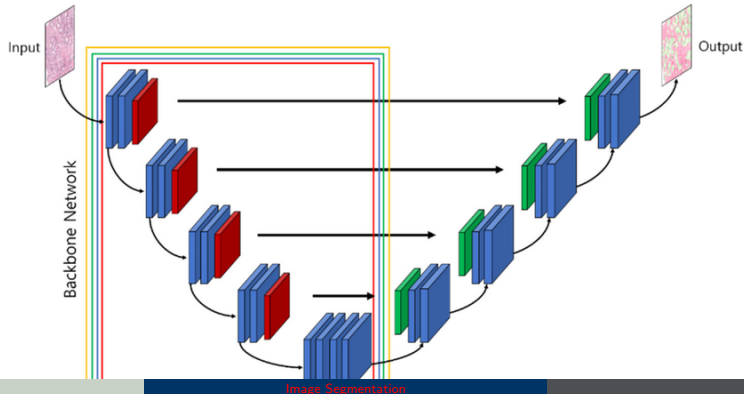
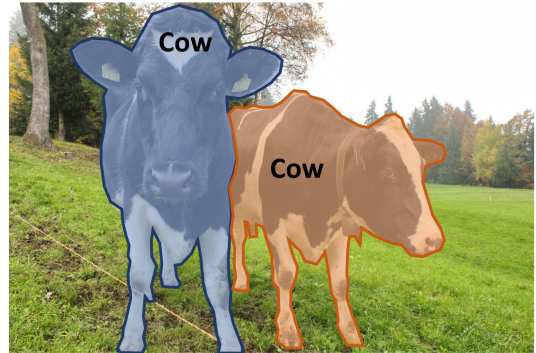
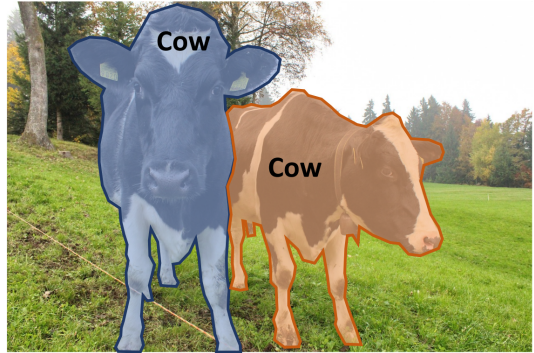


Image Segmentation: **Instance Segmentation**

- Detect all objects in the image, and identify the pixels that belong to each object (Only things!)



- ▶ Detect all objects in the image, and identify the pixels that belong to each object (Only things!)
- ▶ **Approach:** Perform object detection, then predict a segmentation mask for each object!

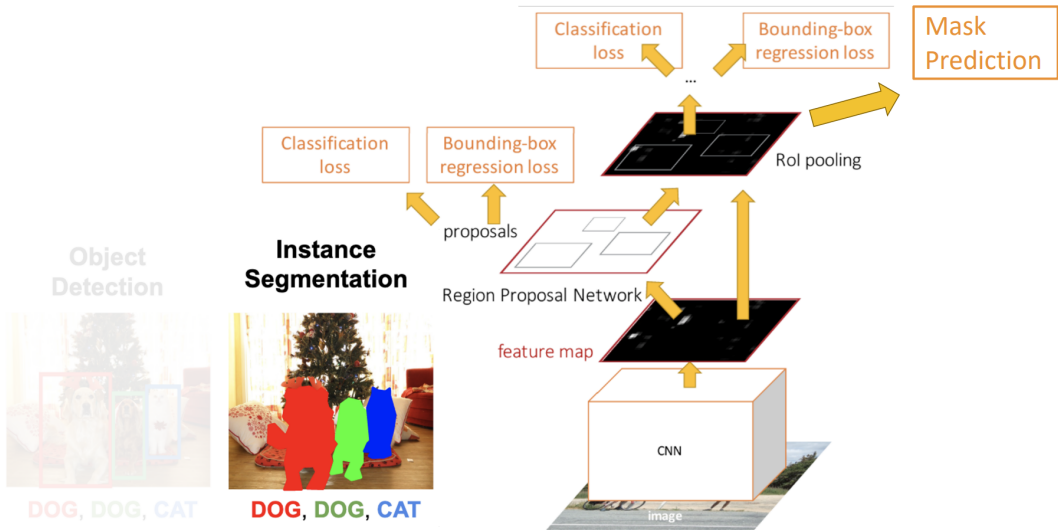


Instance Segmentation: **Mask R-CNN**

Mask R-CNN Overview

- ▶ **Developed on top of Faster R-CNN**
- ▶ Faster R-CNN outputs:
 - Class label
 - Bounding-box offset
- ▶ Mask R-CNN adds a third branch:
 - Predicts object mask for each candidate
- ▶ Performs both **Semantic** and **Instance Segmentation**

Mask R-CNN (cont.)



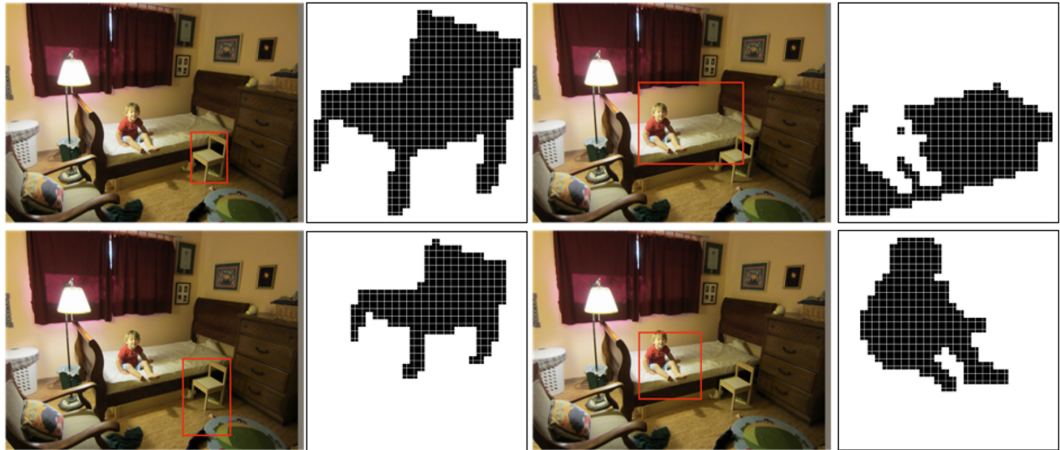
Advantages of Mask R-CNN

- ▶ **Simplicity:** Simple to train
- ▶ **Performance:** Outperforms previous methods, works almost in real-time
- ▶ **Efficiency:** Very efficient, small overhead compared to Faster R-CNN
- ▶ **Flexibility:** Can perform detection and estimation tasks simultaneously

⁰He et al, "Mask R-CNN", ICCV 2017

Mask R-CNN: Example Training Targets

Mask R-CNN: Example Training Targets

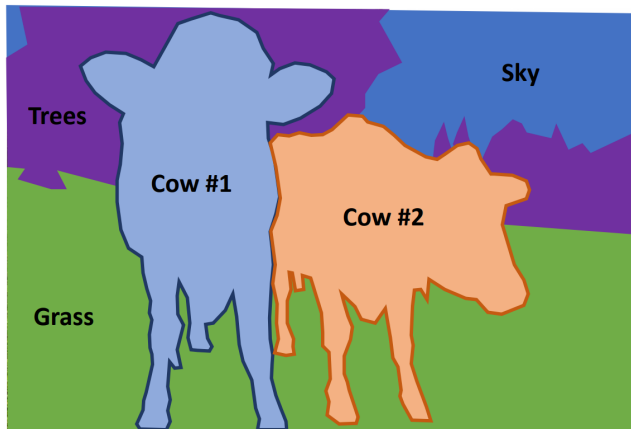


Mask R-CNN: Very Good Results!



Image Segmentation: **Panoptic Segmentation**

- ▶ Label all pixels in the image (both things and stuff)
- ▶ For "thing" categories also separate into instances



Beyond Instance Segmentation: Panoptic Segmentation



⁰Kirillov et al, "Panoptic Feature Pyramid Networks", CVPR 2019

Image Segmentation: **References**

- [1] Long, J., Shelhamer, E., & Darrell, T. (2015).
Fully Convolutional Networks for Semantic Segmentation.
In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
arxiv.org/abs/1411.4038

- [2] Ronneberger, O., Fischer, P., & Brox, T. (2015).
U-Net: Convolutional Networks for Biomedical Image Segmentation.
In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*.
arxiv.org/abs/1505.04597

- [3] He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2017).

Mask R-CNN.

In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

arxiv.org/abs/1703.06870

- [4] Kirillov, A., He, K., Girshick, R., Rother, C., & Dollár, P. (2019).

Panoptic Segmentation.

In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

arxiv.org/abs/1801.00868

- [5] Strudel, R., Garcia, R., Laptev, I., & Schmid, C. (2021).

Segmenter: Transformer for Semantic Segmentation.

In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

arxiv.org/abs/2105.05633

- [6] Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J. M., & Luo, P. (2022).

SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers.

In *Advances in Neural Information Processing Systems (NeurIPS)*.

arxiv.org/abs/2105.15203

Additional Resources:

- PapersWithCode: paperswithcode.com/task/image-segmentation

- ▶ Stanford CS231n Slides (Segmentation Modules): cs231n.stanford.edu/slides/2022/
- ▶ DeepMind Perceiver IO: arxiv.org/abs/2107.14795
- ▶ Google Mask2Former: arxiv.org/abs/2112.01527